

SAOR: Single-View Articulated Object Reconstruction

Mehmet Aygün Oisín Mac Aodha

University of Edinburgh

mehmetaygun.github.io/saor

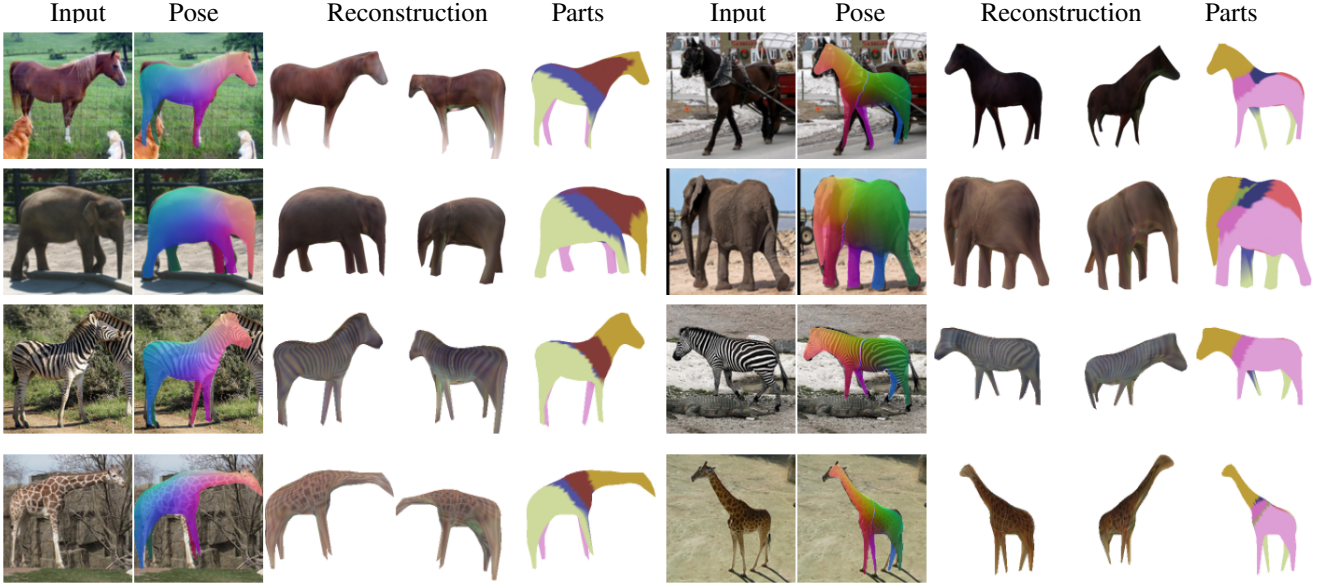


Figure 1: Our SAOR approach is capable of predicting the 3D shape of an articulated object category from a single image. Our per-category models are trained using self-supervision on single-view image collections and can efficiently predict object pose, 3D shape reconstruction, and unsupervised part-level assignment using only a single forward pass per image at test time.

Abstract

We introduce SAOR, a novel approach for estimating the 3D shape, texture, and viewpoint of an articulated object from a single image captured in the wild. Unlike prior approaches that rely on pre-defined category-specific 3D templates or tailored 3D skeletons, SAOR learns to articulate shapes from single-view image collections with a skeleton-free part-based model without requiring any 3D object shape priors. To prevent ill-posed solutions, we propose a cross-instance consistency loss that exploits disentangled object shape deformation and articulation. This is helped by a new silhouette-based sampling mechanism to enhance viewpoint diversity during training. Our method only requires estimated object silhouettes and relative depth maps from off-the-shelf pre-trained networks during training. At inference time, given a single-view image, it efficiently outputs an explicit mesh representation. We obtain improved qualitative and quantitative results on challenging quadruped animals compared to relevant existing work.

1. Introduction

Considered as one of the first PhD theses in computer vision, Roberts [48] aimed to reconstruct 3D objects from single-view images. Despite significant progress in the preceding sixty years [4, 5, 19, 18], the problem remains highly challenging, especially for highly deformable categories photographed in the wild, e.g. animals. In contrast, humans can infer the 3D shape of an object from a single image by making use of priors about the natural world and familiarity with the object category present. Some of these natural-world low-level priors can be explicitly defined (e.g. symmetry or smoothness), but manually encoding and utilizing high-level priors (e.g. 3D category shape templates) for all categories of interest is not a straightforward task.

Recently, multiple methods have attempted to learn 3D shape by making use of advances in deep learning and progress in differentiable rendering [36, 20, 34]. This has resulted in impressive results for synthetic man-made cate-

gories [6, 20, 58] and humans [35, 11], where full or partial 3D supervision is readily available. However, when 3D supervision is not available, the reconstruction of articulated object classes remains challenging. This is due to factors such as: (i) methods not modeling articulation [18, 27, 9, 40], (ii) the reliance on category-specific 3D template [24, 26, 69] or manually defined 3D skeleton supervision [59, 60], or (iii) requiring multi-view training data such as video [59, 24, 61].

In this paper, we introduce **SAOR**, a novel self-supervised **Single-view Articulated Object Reconstruction** method that can estimate the 3D shape of articulating object categories, e.g. animals. We forgo the need for explicit 3D object shape or skeleton supervision at training time by making use of the following assumption: *objects are made of parts, and these parts move together*. Given a single input image, our proposed method predicts the 3D shape of the object and partitions it into parts. It also predicts the transformation for each part and deforms the initially estimated shape, in a skeleton-free manner, using a linear skinning approach. We only require easy to obtain information derived from single-view images during training, e.g. estimated object silhouettes and predicted relative depth maps. SAOR is trained end-to-end, and outputs articulated 3D object shape, texture, 3D part assignment, and camera viewpoint. Example qualitative results can be seen in Fig. 1.

We make the following contributions: (i) We demonstrate that articulation can be learned using image-based self-supervision via our new part-based SAOR approach that does not require any 3D template or object skeleton information during training. (ii) As estimating the 3D shape of an articulated object from a single image is an under-constrained problem, we introduce a cross-instance swap consistency loss that leverages our disentanglement of shape deformation and articulation, in addition to a new silhouette-based sampling mechanism that enhances the diversity of object viewpoints sampled during training. (iii) We illustrate the effectiveness of our approach on a diverse set of challenging quadruped categories, in addition to birds, and present quantitative results where we outperform existing methods that do not require explicit 3D supervision. Code will be made available.

2. Related Work

Here we discuss works that attempt to estimate the 3D shape of an object in a single image using image-based 2D supervision during training. We do not focus on works that require explicit 3D supervision [6, 20, 58, 37] or multi-view images during training [66, 17]. We also do not cover methods that only reconstruct single object instances [38, 43, 44] or models for multi-object scenes [42]. For a recent broad overview of related topics, we refer readers to [54].

Deformable 3D Models. The pioneering work of Blanz

and Vetter [4] marked the introduction of deformable models to represent the 3D shape of an object category using vector spaces. By using 3D scans of human faces, they created a deformable model which captured inter-subject shape variation and demonstrated the ability to reconstruct 3D faces from unseen single-view images. This concept was later expanded to more complex shapes such as the human body [35, 1], hands [52, 21], and animals [70].

Recent work has combined deep learning with 3D deformable models [35, 69, 3, 49] to predict the shape of articulated objects from single-view input images. Given an input image, these methods estimate the parameters of a known deformable 3D model and render the object using the predicted camera viewpoint. Although this line of work has led to impressive results for the human body [35], the results for deformable animal categories are lacking [69, 3, 49]. This is because popular human deformable models, e.g. SMPL [35], are constructed using thousands of high-quality real human 3D scans. In contrast, animal focused 3D models, e.g. SMAL [69], are generated using 3D scans from a small number of toy animals.

These models are parameter-efficient due to their low dimensional shape parameterization, which facilitates easier optimization. However, beyond common categories, such as dogs [49], it can be prohibitively difficult to find 3D scans for each new object category of interest. In this work, we eliminate the need for prior 3D scans of objects by combining linear vertex deformation with a skeleton-free [32] linear blend skinning [29] approach to model the 3D shape of articulated objects using only images at training time.

Unsupervised Learning of 3D Shape. To overcome the need for large collections of aligned 3D scans from an object category of interest, there has been a growing body of work that attempts to learn 3D shape using images from only minimal, if any, 3D supervision. The common theme of these methods is that they treat shape estimation as an image synthesis task during training while enforcing geometric constraints on the rendering process.

One of the first object-centric deep learning-based methods to not use dense 3D shape supervision for single-view reconstruction was CMR [18]. CMR utilizes camera pose supervision estimated from structure from motion, along with human-provided 2D semantic keypoint supervision during training and a coarse template mesh initialized from the keypoints. Subsequently, U-CMR [9] removed the keypoint supervision by using a multi-camera hypothesis approach which assigns and optimizes multiple cameras for each instance during training. IMR [55] starts from a category-level 3D template and learns to estimate shape and camera viewpoint from images and segmentations masks. UMR [31] enforces consistency between per-instance unsupervised 2D part segmentations and 3D shape. They do not assume access to a 3D shape template (or keypoints) but

instead learn one via iterative training. SMR [14] also uses object part segmentation from a self-supervised network as weak supervision. Shelf-SS [65] uses a semi-implicit volumetric representation and obtains consistent multi-view reconstructions using generative models similar to [13]. Like us, all of these methods use object silhouettes (i.e. foreground masks) as supervision.

Recently, Unicorn [40] combined curriculum learning with a cross-instance swap loss to help encourage approximate multi-view consistency across object instances when training a reconstruction network without silhouettes. Their swap loss makes use of an online memory bank to select pairs of images that contain similar shape or texture. The pairs are restricted to be observed from different estimated viewpoints. Then a consistency loss is applied which explicitly forces pairs to share the same shape or texture. In essence, this is a form of weak multi-view supervision under the assumption that the shape of the object pair are the same. However, this assumption breaks down for articulating objects. Inspired by this, we propose a more efficient and effective swap loss designed for articulating objects.

There are also approaches that predict a mapping from image pixels to the surface of a 3D object template as in [11, 41]. CSM [27] eliminates the need for large-scale 2D to 3D surface annotations via an unsupervised 2D to 3D cycle consistency loss. The goal of their loss is to minimize the discrepancy between a pixel location and a corresponding 3D surface point that is reprojection based on the estimated camera viewpoint. In contrast, we do not require any 3D templates or manually defined 2D annotations.

Learning Articulated 3D Shape. Most natural object categories are non-rigid and can thus exhibit some form of articulation. This natural shape variation between individual object instances violates the simplifying assumptions made by approaches that do not attempt to model articulation.

A-CSM [26] extends CSM [27] by making the learned mapping articulation aware. Given a 3D template of the object category, they first manually define the parts of the object category and a hierarchy between the parts. Then, given an input image, they predict transformation parameters for each part so they can articulate the initial 3D template before calculating the mapping between the 3D template and the input pixels. Instead of manually defining parts, [24] initialize sparse handling points, predict displacements for these points, and articulate the shape using differentiable Laplacian deformation. However, each of these methods requires a pre-defined 3D template of the object category.

DOVE [59], LASSIE [63], and MagicPony [60] are recent methods that are capable of predicting the 3D geometry of articulated objects without requiring a 3D category template shape. However, they require a predefined category-level 3D skeleton prior in order to model articulating object parts such as legs. While 3D skeletons are easier to

define compared to full 3D shapes, they still need to be provided for each object category of interest and have to be tailored to the specifics of each category, e.g. the trunk of the elephant is not present in other quadrupeds. In the case of MagicPony [60], in addition to the skeleton and its connectivity, per-bone articulation constraints are also provided, which necessitates more manual labor. Additionally, a single skeleton may be insufficient if there are large shape changes exhibited across instances of the category.

MagicPony [60] builds on DOVE [59], by removing the need for explicit video data during training. Inspired by UMR [31], MagicPony makes use of weak correspondence supervision from a pre-trained self-supervised network to enforce pixel-level consistency between 2D images and learned 3D shape. LASSIE [63] is another skeleton-based approach that also uses correspondence information from self-supervised features during training in addition to manually pre-defined part primitives. Like us, they model object parts, but their goal is not to learn a model that can directly predict shape from a single image at test time. Instead, their approach learns instance shape from a set of images via test-time optimization. In recent work, [64] automatically extracts the skeleton from a user-defined canonical image, but still requires test-time optimization.

Finally, we train with single-view image collections, but there are also several works that use video as a data source for modeling articulating objects [30, 59, 61, 62] and other methods that perform expensive test-time optimization for fitting or refinement [69, 25, 30, 60, 63]. In contrast, we only require self-supervision derived from single-view images and our inference step is performed efficiently via a single forward pass through a deep network.

3. Method

Our objective is to estimate the shape S , texture T , and camera pose (i.e. viewpoint) P of an object from an input image I . To accomplish this, we employ a self-supervised analysis-by-synthesis framework [10, 28] which reconstructs images using a differentiable rendering operation, denoted as $\hat{I} = \Pi(S, T, P)$. The model is optimized by minimizing the discrepancy between a real image I and the corresponding rendered one $\hat{I}$. In this section, we describe how the above quantities are estimated to ensure that the estimated 3D shape is plausible. An overview of the generation phase of our method can be seen in Fig. 2

3.1. SAOR Model

Taking inspiration from previous works [18, 65, 40], we initialize a sphere-shaped mesh with initial vertices S° with fixed connectivity. We then extract a global image representation $\phi_{im} = f_{enc}(I) \in \mathbb{R}^D$ using a neural network encoding function. From this, we utilize several modules, described below, to predict the shape deformation, articulation, camera viewpoint, and object texture necessary to

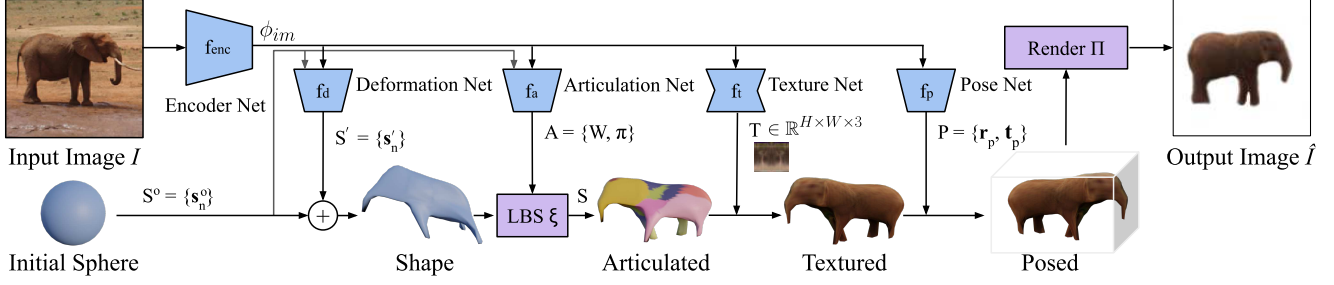


Figure 2: Overview of the generation phase of our SAOR method. Given a single image I as input, we extract a global feature vector ϕ_{im} which is decoded by four separate networks (f_d , f_a , f_t , and f_p) to generate a final output image $\hat{I}$. We start by deforming an initial sphere, articulate it using a part-based linear blend skinning (LBS) operation ξ , texture the mesh, and render it using a differential render Π so that it is depicted from the same viewpoint as the input image. The parameters for each of the networks presented are trained in an end-to-end manner using image reconstruction-based self-supervision.

generate the final target shape.

Shape. We predict the object shape by deforming and articulating an initial sphere mesh $S^o = \{s_n^o\}_n^N$. Here, each of the N elements of S^o is a 3D coordinate. We estimate the vertices of the deformed shape using a deformation function $s'_i = s_i^o + f_d(s_i^o, \phi_{im})$, which outputs the displacement vector for the initial points. The deformation function f_d is modeled as a functional field, which is a 3-layer MLP similar to [42, 40]. As most natural objects exhibit bilateral symmetry, we only deform the vertices of the zero-centered initial shape that are located on the positive side of the xy -plane and reflect the deformation for the vertices on the negative side similar to [18]. We then articulate the deformed shape using linear skinning [29] in a skeleton-free manner [32] to obtain the final shape $S = \xi(S', A)$, where A is the output of our articulation prediction function, which we describe in more detail later in Sec. 3.2.

Texture. To predict the texture of the object, we generate a UV image by transforming the global image feature, $T = f_t(\phi_{im})$. The function f_t is implemented as a convolutional decoder, which maps a one-dimensional input representation to a texture map, $f_t : \mathbb{R}^D \mapsto \mathbb{R}^{H \times W \times 3}$. This approach is similar to previous works [40, 42]. However, unlike existing work [18, 31] that copy the pixel colors of the input image directly to create a texture image using a predicted flow field, we predict texture directly. In initial experiments, we found that estimating texture flow only gave minimal improvements, for an increase in complexity.

Camera Pose. We use Euler angles (azimuth, elevation, and roll) along with camera translation to predict the camera pose, similar to previous works [9, 40]. Instead of using multiple camera hypotheses for each input instance [40], for each forward pass, or optimizing them for each training instance [9], we use several camera pose predictors, but only select the one with the highest confidence score for each forward pass, as described in [60]. Specifically, we predict the camera pose as $P \in \mathbb{R}^6 = f_p(\phi_{im})$. Here, $P = \{r_p, t_p\}$ represents the predicted camera rotation and translation. This

approach accelerates the training process and reduces memory requirements since we only need to compute the loss for one camera in each forward pass. We only incorporate priors about the ranges of elevation and roll predictions, instead of a strong uniformity constraint on the distribution of the camera poses as in [40].

3.2. Skeleton-Free Articulation

Many natural world object categories exhibit some form of articulation, e.g. the legs of an animal. Existing work has attempted to model this via deformable 3D template models [49] or by using manually defined category-level skeleton priors [59, 60]. However, this assumes one has access to category-level 3D supervision during training. We instead propose a skeleton-free approach by modeling articulation using a part-based model. Our approach is inspired by [32], who proposed a related skeleton-free representation for the task of pose transfer between 3D meshes. However, in our case we train a model that can predict parts in an image from self-supervision alone.

The core idea is to partition the 3D shape into parts and deform each part based on predicted transformations. To achieve this, we predict a part assignment matrix $W \in \mathbb{R}^{N \times K}$, that represents how likely it is that a vertex belongs to a particular part, where $\sum_k^K W_{i,k} = 1$. Here, K is a hyperparameter that represents the number of parts and N is the number of vertices in the mesh. We also predict transformation parameters $\pi = \{(z_k, r_k, t_k)\}_k^K$ for each part which consists of scale $z \in \mathbb{R}^3$, rotation $r \in \mathbb{R}^{3 \times 3}$, and translation $t \in \mathbb{R}^3$. Each of these parameters are predicted using different MLPs that take the global image feature ϕ_{im} as input and output $A = \{W, \pi\} = f_a(S^o, \phi_{im})$.

Articulation can be applied to a shape using a set of deformations using the linear blend skinning equation [16]. Here, each vertex needs to be associated with deformations by the skinning weights. In previous work [59, 63, 60] skinning weights are calculated using a skeleton prior (e.g. a set of bones and their connectivity). We instead estimate skin-

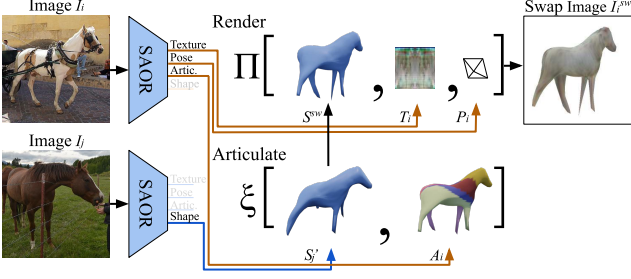


Figure 3: Illustration of our articulated swap loss. To calculate the loss, a swap image $\hat{I}_i^{sw}$ is rendered using a randomly chosen paired image’s shape S'_j , combined with estimated texture, viewpoint, and articulation (T_i, P_i, A_i) from the input image I_i . It ensures that 3D predictions are not degenerate and helps disentangle deformation and articulation.

ning weights using a part-based model that does not require a prior skeleton or any ground truth part segmentations. We first calculate the centers for each part from the vertices of the deformed shape $s'_i \in S'$,

$$c_k = \frac{\sum_i^N s'_i * W_{i,k}}{\sum_i^N W_{i,k}}. \quad (1)$$

The final position of a vertex s_i for the final shape S is then calculated using the skinning weight of the vertex and estimated part transformations as

$$s_i = \sum_k^K W_{i,k} z_k \odot (r_k(s'_i - c_k) + t_k), \quad (2)$$

where z_k , r_k , and t_k are the predicted scale, rotation, and translation parameters corresponding to part k and $\odot$ is an element-wise multiplication. In addition to the reconstruction losses, we apply regularization on the part assignment matrix W that encourages the size of each part segment to be similar for each instance. As each of the above operations are differentiable, articulation is learned via self-supervised without requiring any 3D template shapes [26], predefined skeletons [60], or part segmentations [31].

3.3. Swap Loss and Balanced Sampling

One of the hardest challenges in single-view 3D reconstruction is the tendency to predict degenerate solutions as a result of the ill-posed nature of the task (i.e. an infinite number of 3D shapes can explain the same 2D input). Examples of such failure cases include models predicting flat 2D textured planes which are visually consistent when viewed from the same pose as the input image but lack full 3D shape [40]. To mitigate these issues, and to ensure multi-view consistency of our 3D reconstructions, we build on the swap loss idea recently introduced in [40].

To estimate their swap loss, [40] take a pair of images (I_i, I_j) that depict two different instances of the same ob-

ject category, and estimate their respective shape, texture, and camera pose, ($\{S_i, T_i, P_i\}, \{S_j, T_j, P_j\}$). They then generate an image $\hat{I}_i^{sw} = \Pi(S_j, T_i, P_i)$ by swapping the shape encodings S_i and S_j , where Π is a differentiable renderer. Finally, they estimate the appearance loss between I_i and $\hat{I}_i^{sw}$ which aims to enforce cross-instance consistency. The intuition here is that the shape from I_j and texture from I_i should be sufficient to describe the appearance of I_i , even though I_j is potentially captured from a different viewpoint.

In [40], the shapes S_i and S_j should be similar, while the predicted viewpoints P_i and P_j should be different to get a useful ‘multi-view’ training signal. To obtain similar shapes, they store latent shape codes in a memory bank which is queried online via a nearest neighbor lookup. This memory bank is updated at each iteration for the selected shape codes using the current state of the network. Moreover, they limit the search neighborhood based on the predicted viewpoints to ensure that they obtain some viewpoint variations, i.e. in [40] the viewpoints P_i and P_j should not be too similar, or too different. While this results in good results from some mostly rigid categories such as birds and cars, for highly articulated animal categories it can lead to degenerate solutions, as can be seen in Fig. 7.

Swap Loss. To address this issue, we introduced a straightforward but more effective swap loss that generalizes to articulated object classes. Our hypothesis is that given a set of images that contain a variety of viewpoints exhibiting disentangled deformation and articulation, we can use randomly chosen image pairs to calculate the swap loss. Since we model the articulation along with the deformation to obtain the final shape, articulation can be used to explain the difference between shapes. In our proposed loss, we swap random deformed shapes S'_i and S'_j but use the original estimated articulation $S^{sw} = \xi(S'_j, A_i)$ and reconstruct the swap image $\hat{I}_i^{sw} = \Pi(S^{sw}, T_i, P_i)$ to calculate the swap loss $\mathcal{L}_{swap}(I_i, \hat{I}_i^{sw})$. Our loss is illustrated in Fig. 3.

Balanced Sampling. For the swap loss to be successful it requires the selected image pairs to be from different viewpoints. To obtain informative image pairs, we propose an image sampling mechanism which makes use of the segmentation masks of the input images. Before training, we cluster predicted segmentation masks of the training images and then during training we sample images from each cluster uniformly to form batches. This ensures that each batch includes the object of interest depicted from different viewpoints. In Fig. 4 we can see that cluster centers mostly capture the rough distribution of viewpoints and thus help stabilize training. As our image pairs (I_i, I_j) are sampled from within the same batch during training, this results in varied images from different viewpoints for the swap loss. Our swap and balanced sampling steps combined drastically simplifies the swap loss from [40] and improve performance and training stability on articulated classes.

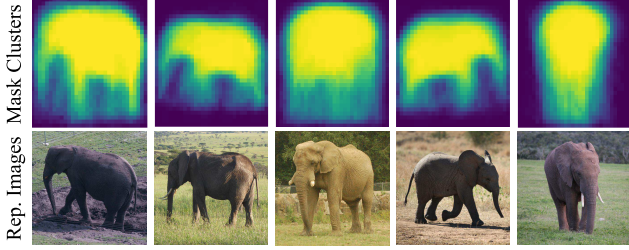


Figure 4: (Top) Subset of the resulting cluster centers that arise from clustering the object segmentation masks. (Bottom) Representative images from each of the corresponding clusters above. We can see that our simple clustering operation captures the main viewpoint variations present in the data, e.g. left facing, frontal, right facing, *etc.*

3.4. Optimization

Given an input image, I , we reconstruct it as $\hat{I}$ using estimated shape, texture, and viewpoint. In addition, we use the swapped shape to predict another image $\hat{I}^{sw}$ and calculate the swap loss, as discussed in Sec. 3.3. We also use differentiable rendering to obtain a predicted object segmentation mask and depth derived from the predicted 3D shape, $\hat{M}$ and $\hat{D}$ respectively. Our model is trained using a combination of the following losses,

$$\mathcal{L} = \mathcal{L}_{appr} + \mathcal{L}_{mask} + \mathcal{L}_{depth} + \mathcal{L}_{swap} + \mathcal{L}_{reg}. \quad (3)$$

The appearance loss, $\mathcal{L}_{appr}(I, \hat{I})$, uses an RGB and perceptual loss [68], $\mathcal{L}_{mask}(M, \hat{M})$ estimates silhouette discrepancy, and $\mathcal{L}_{depth}(D, \hat{D})$ is the translation and shift-invariant depth loss introduced in [45]. To avoid degenerate solutions, we use $\mathcal{L}_{swap}(I, \hat{I}^{sw})$ and regularize predictions using $\mathcal{L}_{reg}$, that combines smoothness [7] and normal consistency on the predicted 3D shape along with a uniform distribution on part assignment. While we used predicted segmentation masks and relative depth during training, at test time, our model only requires an image.

3.5. Implementation Details

We employ a ResNet [12] as our global encoder, f_{enc} , and train it end-to-end using Adam [22]. Object masks M and depths D are obtained for training by utilizing off-the-shelf pre-trained networks. To implement all 3D operations in our model we use the Pytorch3D framework [47] using their default mesh rasterization operations [34] which is differentiable and enables end-to-end model training. Prior to being passed to the model, images are resized to 128x128 pixels. We disable articulation for the first 100 epochs when training a model from scratch, and finetune models on other categories models for 200 epochs by learning deformation and articulation jointly. Training on a dataset with 10k images for 500 epochs takes approximately 10 hours using a single NVIDIA RTX-3090 GPU. The lightweight design

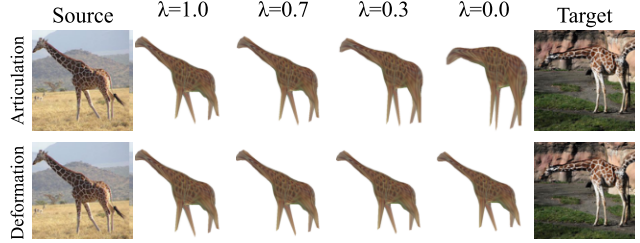


Figure 5: Disentanglement of articulation and deformation. On top, we interpolate articulation latent features between the source and target image, and on the bottom do the same for shape deformation features. $\lambda = 1$ indicates that original features are used for reconstruction, while $\lambda = 0$ indicates the target ones. We can see that the difference between the reconstructions is explained by articulation changes between the source and target image pairs.

of our proposed method enables the estimation of the final shape, articulation, texture, and viewpoint in approximately 15 ms per image. We provide more details about the loss terms, hyperparameters, and optimization in the supplementary material.

4. Experiments

Here we present results on several quadruped animal categories including elephants, horses, zebras, cows, in addition to birds, and include both quantitative and qualitative comparisons to previous work.

4.1. Data and Pre-Processing

For quadruped experiments, we used 10k horse images from the LSUN [67] dataset with an additional 500 front-facing images from iNaturalist [15], as LSUN mostly contains side-view images of horses, to train our initial model. As in [60], we then finetune the horse model on different quadruped categories. For the cow, zebra, and giraffe experiments, we used approximately 1k images from iNaturalist [15] and SD-Elephant [39] for elephants. Bird experiments are performed on CUB [57], where the models are trained from scratch using the original train/test split.

For pre-processing, we run a general-purpose animal object detector [2] to detect all the animals present in the input images and then filter the detections based on the confidence, size, and location of the bounding box. We then extract segmentation masks using PointRend [23] pre-trained on COCO [33] and estimate the relative monocular depth using the transformer-based Midas [45, 46].

4.2. Quantitative Results

To compare to existing work, we quantitatively evaluate using the 2D keypoint transfer task, which reflects the quality of the estimated shape and viewpoint. We report results using the PCK metric with a 0.1 threshold.

Supervision	Method	all	w/o aqua
$\text{cube}^*, \text{mask}, \text{cam}, \text{key}$	CMR [18]	54.6	59.1
$\text{cube}^*, \text{mask}$	U-CMR [9]	35.9	41.2
$\text{key}, \text{mask}^*, \text{cam}, \text{key}^*$	DOVE [59]	44.7	51.0
$\text{key}, \text{mask}^*, \text{key}^*, \dagger$	MagicPony [60]	55.4	63.5
mask	CMR [18]	25.5	27.7
$\text{mask}, \text{SCOPS}$	UMR [31]	51.2	55.5
None	Unicorn [40]	49.0	53.5
mask^*	Ours	49.9	55.9
$\text{mask}^*, \text{depth}^*$	Ours	51.9	57.8

Table 1: Keypoint transfer results on CUB [57] using the PCK metric with 0.1 threshold, where higher numbers are better. cube 3D template shape, key 3D skeleton, cam camera viewpoint, key 2D keypoints, mask segmentation mask, key^* optical flow, cam^* video, key^* DINO features, and depth^* monocular depth. $\dagger$ also uses additional video frames from [59]. The initial 3D template in [18, 9] is derived from 2D keypoints. $*$ indicates that the supervision is predicted, hence a form of weak supervision. We obtain the best results for methods that do not use 3D templates (cube), skeletons (key), or extra data during training in addition to CUB (e.g. [59, 60]).

Birds. Results on CUB [57] are presented in Table 1 for all bird classes along with non-aquatic classes as in [60]. Our method obtains the best results out of methods that do not use keypoint supervision, 3D object priors (e.g. 3D templates or skeletons [59, 60]), or additional data (e.g. [59, 60]). We also report the performance of our model trained without predicted depth supervision, which only obtains a slightly lower score than our full model.

Quadrupeds. Results for quadruped animals from the Pascal dataset [8] are presented in Table 2. As noted earlier, we trained the horse model from scratch, while the other models were finetuned using data from iNaturalist [15]. For the Unicorn [40] baseline, we used their pre-trained model which was also trained on LSUN horses. For the remaining categories, we also finetuned their model in a similar fashion to ours. Our method outperforms CSM [27] and its articulated version A-CSM [26], which use a 3D template of the object category and 3D part segmentation for the horse and cow category. Moreover, our method achieved significantly better scores than Unicorn [40], which produces degenerate (i.e. flat) shape predictions for these classes (see Fig. 7). We visualize some keypoint transfer results in Fig 6.

4.3. Ablation Experiments

To provide insight into the impact of our proposed model components, we provide ablation experiments on CUB in Table 3. While depth information helps to improve results, we can see that our articulation and swap modules are significantly more important. Our model trained without the swap loss obtains reasonable keypoint matching per-

Supervision	Method	Horse	Cow	Sheep
mask	Dense-Equi [53]	23.3	20.9	19.6
mask, cube	CSM [27]	31.2	26.3	24.7
mask, cube	A-CSM [26]	32.9	26.3	28.6
None	Unicorn [40]	14.9	12.1	11.0
mask^*	Ours	32.6	24.7	18.4
$\text{mask}^*, \text{depth}^*$	Ours	34.7	26.9	21.7

Table 2: Keypoint transfer results for quadruped animals. Note, there is no code available for [59, 60] or quantitative results for them on these categories.

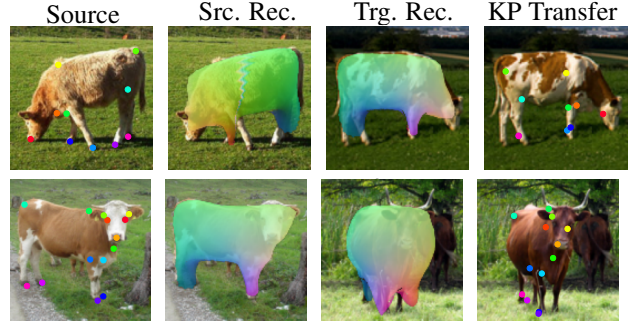


Figure 6: Keypoint transfer results. Our model captures articulation and viewpoint differences between images.

Method	All	w/o aqua
Ours	51.9	57.8
Ours w/o depth	49.9	55.9
Ours w/o swap	44.5	48.4
Ours w/o sampling	38.8	41.9
Ours w/o articulation	41.7	45.8

Table 3: Keypoint transfer ablation results on CUB using the PCK at 0.1 metric. Here we disable individual contributions of our model to measure their impact.

formance but produces degenerate flat plane-like solutions. The performance also drops if articulation is not utilized. This is because we choose random pairs for the swap loss (unlike [40]’s more expensive pair selection), and thus only viewpoint changes can be used to explain the difference between images.

4.4. Qualitative Results

Comparison with Previous Work. We compare SAOR with methods that do not use any 3D shape priors (i.e. Unicorn [40] and UMR [31]) and methods that use a 3D skeleton prior (i.e. DOVE [59] and MagicPony [60]). A comparison of shape estimates on horse images can be seen in Fig. 7 and Fig. 8. While Unicorn produces reasonable reconstructions from the input viewpoint, their predictions are flat from different viewpoints. UMR predicts very thin 3D shapes and does not generate four legs. Our method reconstructs multi-view consistent 3D shapes, with prominent four legs. Dove uses video data during training along with

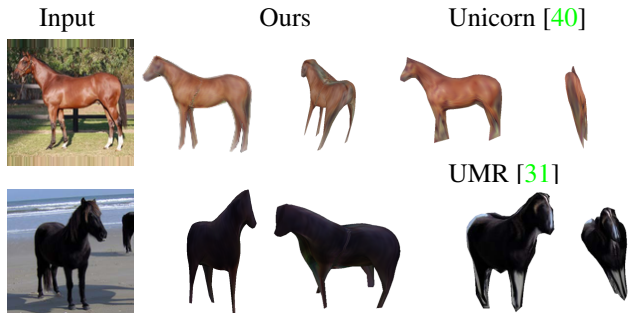


Figure 7: Comparison of our model to Unicorn [40] and UMR [31] on horses. Compared to UMR which predicts thin shapes with two legs, we can reconstruct multi-view consistent results with four legs. Unicorn fails to produce 3D consistent shapes.

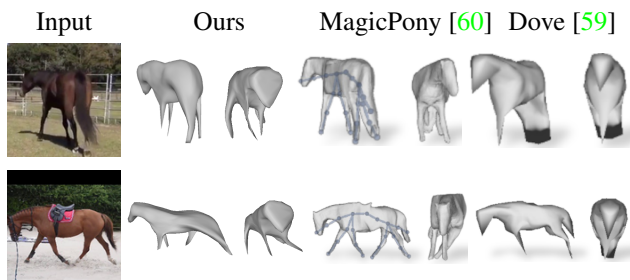


Figure 8: Comparison of our model to MagicPony [60] and Dove [59], which both use a skeleton prior during training. Compared to Dove, we able to predict four legs and only obtain slightly worse reconstructions compared to MagicPony without using any 3D prior about the articulation of the object class and with a simpler and more efficient architecture.

a skeleton prior but still fails to reconstruct details like all four legs. In general, MagicPony produces better shapes with more detail than us. However, its hybrid volumetric-mesh representation requires an extra transformation from implicit to explicit representation using [50] and requires multiple rendering operations to estimate the final shape.

Deformation and Articulation Disentanglement. In Fig. 5 we illustrate the disentanglement of articulation and deformation learned by our model. Given two images depicting differently articulating instances, we interpolate the deformation and articulation features between them to visualize reconstructions. While interpolating the articulation feature changes the result, changing the deformation feature does not as the shape difference between both images can be explained via articulation changes.

Part Consistency. After finetuning the pre-trained horse model on different quadruped categories, we observe that the predicted part assignments stay consistent across categories, as can be seen in Fig. 1. For instance, although the shapes of giraffes and elephants are significantly different, our method is able to assign similar parts to similarly articulated areas. Here, each color represents the part that is

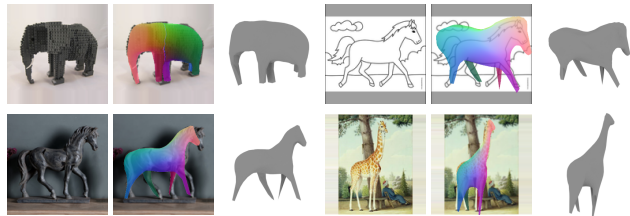


Figure 9: Our model, trained on real-world images, plausibly estimates the 3D shape and viewpoint of images from different domains (e.g. lego, line drawings, and paintings).

predicted with the highest probability from the part assignment matrix W by the articulation network f_a .

Out-of-Distribution Images. We illustrate the generalization capabilities of our method by predicting 3D shapes from non-photoreal images, e.g. drawings. In Fig. 9 we observe that our method is able to reconstruct plausible shapes and poses of animals from input images that are very different from the training domain.

4.5. Discussion and Limitations

Although our proposed method is able to estimate detailed 3D shapes, the texture predictions lack detail and realism. This could be improved using test-time refinement similar to [60] or alternative texture representations. During training, our method uses estimated silhouettes and relative depth maps as supervision. Depth maps come from a generic pre-trained model, hence are free to acquire as the model is trained on completely different scene-level datasets [46]. However, the segmentation model [23] needs to be trained on specific categories. Moreover, our method fails to generate good shapes if the input images contains unusual viewpoints that differ significantly from the training images. We present some examples of failure cases in the supplementary material.

5. Conclusion

We presented SAOR, a new approach for single-view articulated object reconstruction. Our method is capable of predicting the 3D shape of articulated object categories without requiring any explicit object specific 3D information, e.g. 3D templates or skeletons, at training time. To achieve this, we learn to segment objects into parts which move together and propose a new swap-based regularization loss that improves 3D shape consistency in addition to simplifying training compared to competing methods.

Currently, we have a separate model for each category. Given that SAOR does not require manually defined 3D category-level priors, in future work we intend to explore the joint setting by scaling to multiple articulating categories in the same model [56].

Acknowledgments: OMA was in part supported by university partner contributions to the Alan Turing Institute.

References

- [1] Dragomir Anguelov, Praveen Srinivasan, Daphne Koller, Sebastian Thrun, Jim Rodgers, and James Davis. Scape: shape completion and animation of people. In *ACM SIGGRAPH*, 2005. 2
- [2] Sara Beery, Dan Morris, and Siyu Yang. Efficient pipeline for camera trap image review. *arXiv:1907.06772*, 2019. 6, 12
- [3] Benjamin Biggs, Oliver Boyne, James Charles, Andrew Fitzgibbon, and Roberto Cipolla. Who left the dogs out? 3d animal reconstruction with expectation maximization in the loop. In *ECCV*, 2020. 2
- [4] Volker Blanz and Thomas Vetter. A morphable model for the synthesis of 3d faces. In *ACM SIGGRAPH*, 1999. 1, 2
- [5] Thomas J Cashman and Andrew Fitzgibbon. What shape are dolphins? building 3d morphable models from 2d images. *TPAMI*, 2012. 1
- [6] Christopher B Choy, Danfei Xu, JunYoung Gwak, Kevin Chen, and Silvio Savarese. 3d-r2n2: A unified approach for single and multi-view 3d object reconstruction. In *ECCV*, 2016. 2
- [7] Mathieu Desbrun, Mark Meyer, Peter Schröder, and Alan H Barr. Implicit fairing of irregular meshes using diffusion and curvature flow. In *ACM SIGGRAPH*, 1999. 6
- [8] Mark Everingham, SM Ali Eslami, Luc Van Gool, Christopher KI Williams, John Winn, and Andrew Zisserman. The pascal visual object classes challenge: A retrospective. In *IJCV*, 2015. 7, 12
- [9] Shubham Goel, Angjoo Kanazawa, and Jitendra Malik. Shape and viewpoint without keypoints. In *ECCV*, 2020. 2, 4, 7
- [10] U. Grenander. *Lectures in Pattern Theory: Volume 2 Pattern Analysis*. 1978. 3
- [11] Rıza Alp Güler, Natalia Neverova, and Iasonas Kokkinos. Densepose: Dense human pose estimation in the wild. In *CVPR*, 2018. 2, 3
- [12] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016. 6, 12
- [13] Philipp Henzler, Niloy J Mitra, and Tobias Ritschel. Escaping plato’s cave: 3d shape from adversarial rendering. In *ICCV*, 2019. 3
- [14] Tao Hu, Liwei Wang, Xiaogang Xu, Shu Liu, and Jiaya Jia. Self-supervised 3d mesh reconstruction from single images. In *CVPR*, 2021. 3
- [15] iNaturalist. iNaturalist. www.inaturalist.org, accessed 8 March 2023. 6, 7
- [16] Alec Jacobson, Zhigang Deng, Ladislav Kavan, and John P Lewis. Skinning: Real-time shape deformation. In *ACM SIGGRAPH Courses*. 2014. 4
- [17] Ajay Jain, Matthew Tancik, and Pieter Abbeel. Putting nerf on a diet: Semantically consistent few-shot view synthesis. In *ICCV*, 2021. 2
- [18] Angjoo Kanazawa, Shubham Tulsiani, Alexei A Efros, and Jitendra Malik. Learning category-specific mesh reconstruction from image collections. In *ECCV*, 2018. 1, 2, 3, 4, 7
- [19] Abhishek Kar, Shubham Tulsiani, Joao Carreira, and Jitendra Malik. Category-specific object reconstruction from a single image. In *CVPR*, 2015. 1
- [20] Hiroharu Kato, Yoshitaka Ushiku, and Tatsuya Harada. Neural 3d mesh renderer. In *CVPR*, 2018. 1, 2
- [21] Sameh Khamis, Jonathan Taylor, Jamie Shotton, Cem Keskin, Shahram Izadi, and Andrew Fitzgibbon. Learning an efficient model of hand shape variation from depth images. In *CVPR*, 2015. 2
- [22] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv:1412.6980*, 2014. 6, 15
- [23] Alexander Kirillov, Yuxin Wu, Kaiming He, and Ross Girshick. Pointrend: Image segmentation as rendering. In *CVPR*, 2020. 6, 8, 12
- [24] Filippos Kokkinos and Iasonas Kokkinos. Learning monocular 3d reconstruction of articulated categories from motion. In *CVPR*, 2021. 2, 3
- [25] Filippos Kokkinos and Iasonas Kokkinos. To the point: Correspondence-driven monocular 3d category reconstruction. *NeurIPS*, 2021. 3
- [26] Nilesh Kulkarni, Abhinav Gupta, David Fouhey, and Shubham Tulsiani. Articulation-aware canonical surface mapping. In *CVPR*, 2020. 2, 3, 5, 7, 12
- [27] Nilesh Kulkarni, Abhinav Gupta, and Shubham Tulsiani. Canonical surface mapping via geometric cycle consistency. In *ICCV*, 2019. 2, 3, 7
- [28] Tejas D Kulkarni, William F Whitney, Pushmeet Kohli, and Josh Tenenbaum. Deep convolutional inverse graphics network. In *NeurIPS*, 2015. 3
- [29] John P Lewis, Matt Cordner, and Nickson Fong. Pose space deformation: a unified approach to shape interpolation and skeleton-driven deformation. In *ACM SIGGRAPH*, 2000. 2, 4
- [30] Xueting Li, Sifei Liu, Shalini De Mello, Kihwan Kim, Xiaolong Wang, Ming-Hsuan Yang, and Jan Kautz. Online adaptation for consistent mesh reconstruction in the wild. *NeurIPS*, 2020. 3
- [31] Xueting Li, Sifei Liu, Kihwan Kim, Shalini De Mello, Varun Jampani, Ming-Hsuan Yang, and Jan Kautz. Self-supervised single-view 3d reconstruction via semantic consistency. In *ECCV*, 2020. 2, 3, 4, 5, 7, 8
- [32] Zhouyingcheng Liao, Jimei Yang, Jun Saito, Gerard Pons-Moll, and Yang Zhou. Skeleton-free pose transfer for stylized 3d characters. In *ECCV*, 2022. 2, 4
- [33] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *ECCV*, 2014. 6, 12
- [34] Shichen Liu, Tianye Li, Weikai Chen, and Hao Li. Soft rasterizer: A differentiable renderer for image-based 3d reasoning. In *ICCV*, 2019. 1, 6
- [35] Matthew Loper, Naureen Mahmood, Javier Romero, Gerard Pons-Moll, and Michael J Black. Smpl: A skinned multi-person linear model. *TOG*, 2015. 2
- [36] Matthew M Loper and Michael J Black. Opendr: An approximate differentiable renderer. In *ECCV*, 2014. 1

- [37] Lars Mescheder, Michael Oechsle, Michael Niemeyer, Sebastian Nowozin, and Andreas Geiger. Occupancy networks: Learning 3d reconstruction in function space. In *CVPR*, 2019. 2
- [38] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 2021. 2
- [39] Ron Mokady, Omer Tov, Michal Yarom, Oran Lang, Inbar Mosseri, Tali Dekel, Daniel Cohen-Or, and Michal Irani. Self-distilled stylegan: Towards generation from internet photos. In *ACM SIGGRAPH*, 2022. 6
- [40] Tom Monnier, Matthew Fisher, Alexei A Efros, and Mathieu Aubry. Share with thy neighbors: Single-view reconstruction by cross-instance consistency. In *ECCV*, 2022. 2, 3, 4, 5, 7, 8, 15
- [41] Natalia Neverova, David Novotný, and Andrea Vedaldi. Continuous surface embeddings. In *NeurIPS*, 2020. 3
- [42] Michael Niemeyer and Andreas Geiger. Giraffe: Representing scenes as compositional generative neural feature fields. In *CVPR*, 2021. 2, 4
- [43] Keunhong Park, Utkarsh Sinha, Jonathan T Barron, Sofien Bouaziz, Dan B Goldman, Steven M Seitz, and Ricardo Martin-Brualla. Nerfies: Deformable neural radiance fields. In *ICCV*, 2021. 2
- [44] Ben Poole, Ajay Jain, Jonathan T Barron, and Ben Mildenhall. Dreamfusion: Text-to-3d using 2d diffusion. In *ICLR*, 2023. 2
- [45] René Ranftl, Alexey Bochkovskiy, and Vladlen Koltun. Vision transformers for dense prediction. *ICCV*, 2021. 6, 12
- [46] René Ranftl, Katrin Lasinger, David Hafner, Konrad Schindler, and Vladlen Koltun. Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer. *TPAMI*, 2022. 6, 8, 12
- [47] Nikhila Ravi, Jeremy Reizenstein, David Novotny, Taylor Gordon, Wan-Yen Lo, Justin Johnson, and Georgia Gkioxari. Accelerating 3d deep learning with pytorch3d. *arXiv:2007.08501*, 2020. 6
- [48] Lawrence G Roberts. *Machine perception of three-dimensional solids*. PhD thesis, Massachusetts Institute of Technology, 1963. 1
- [49] Nadine Rueegg, Silvia Zuffi, Konrad Schindler, and Michael J Black. Barc: Learning to regress 3d dog shape from images by exploiting breed information. In *CVPR*, 2022. 2, 4
- [50] Tianchang Shen, Jun Gao, Kangxue Yin, Ming-Yu Liu, and Sanja Fidler. Deep marching tetrahedra: a hybrid representation for high-resolution 3d shape synthesis. *NeurIPS*, 2021. 8
- [51] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *ICLR*, 2015. 13
- [52] Jonathan Taylor, Richard Stebbing, Varun Ramakrishna, Cem Keskin, Jamie Shotton, Shahram Izadi, Aaron Hertzmann, and Andrew Fitzgibbon. User-specific hand modeling from monocular depth sequences. In *CVPR*, 2014. 2
- [53] James Thewlis, Hakan Bilen, and Andrea Vedaldi. Unsupervised learning of object frames by dense equivariant image labelling. In *NeurIPS*, 2017. 7
- [54] Edith Tretschk, Navami Kairanda, Mallikarjun B R, Rishabh Dabral, Adam Kortylewski, Bernhard Egger, Marc Habermann, Pascal Fua, Christian Theobalt, and Vladislav Golyanik. State of the art in dense monocular non-rigid 3d reconstruction. *arXiv:2210.15664*, 2022. 2
- [55] Shubham Tulsiani, Nilesch Kulkarni, and Abhinav Gupta. Implicit mesh reconstruction from unannotated image collections. *arXiv:2007.08504*, 2020. 2
- [56] Kalyan Alwala Vasudev, Abhinav Gupta, and Shubham Tulsiani. Pre-train, self-train, distill: A simple recipe for super-sizing 3d reconstruction. In *CVPR*, 2022. 8
- [57] Catherine Wah, Steve Branson, Peter Welinder, Pietro Perona, and Serge Belongie. The caltech-ucsd birds-200-2011 dataset. 2011. 6, 7, 12
- [58] Nanyang Wang, Yinda Zhang, Zhuwen Li, Yanwei Fu, Wei Liu, and Yu-Gang Jiang. Pixel2mesh: Generating 3d mesh models from single rgb images. In *ECCV*, 2018. 2
- [59] Shangzhe Wu, Tomas Jakab, Christian Rupprecht, and Andrea Vedaldi. Dove: Learning deformable 3d objects by watching videos. *arXiv:2107.10844*, 2021. 2, 3, 4, 7, 8
- [60] Shangzhe Wu, Ruining Li, Tomas Jakab, Christian Rupprecht, and Andrea Vedaldi. Magicpony: Learning articulated 3d animals in the wild. In *CVPR*, 2023. 2, 3, 4, 5, 6, 7, 8, 12, 13, 15
- [61] Gengshan Yang, Deqing Sun, Varun Jampani, Daniel Vlasic, Forrester Cole, Huiwen Chang, Deva Ramanan, William T Freeman, and Ce Liu. Lasr: Learning articulated shape reconstruction from a monocular video. In *CVPR*, 2021. 2, 3
- [62] Gengshan Yang, Deqing Sun, Varun Jampani, Daniel Vlasic, Forrester Cole, Ce Liu, and Deva Ramanan. Viser: Video-specific surface embeddings for articulated 3d shape reconstruction. *NeurIPS*, 2021. 3
- [63] Chun-Han Yao, Wei-Chih Hung, Yuanzhen Li, Michael Rubinstein, Ming-Hsuan Yang, and Varun Jampani. Lassie: Learning articulated shape from sparse image ensemble via 3d part discovery. In *NeurIPS*, 2022. 3, 4
- [64] Chun-Han Yao, Wei-Chih Hung, Yuanzhen Li, Michael Rubinstein, Ming-Hsuan Yang, and Varun Jampani. Hi-lassie: High-fidelity articulated shape and skeleton discovery from sparse image ensemble. In *CVPR*, 2023. 3
- [65] Yufei Ye, Shubham Tulsiani, and Abhinav Gupta. Shelf-supervised mesh prediction in the wild. In *CVPR*, 2021. 3
- [66] Alex Yu, Vickie Ye, Matthew Tancik, and Angjoo Kanazawa. pixelnerf: Neural radiance fields from one or few images. In *CVPR*, 2021. 2
- [67] Fisher Yu, Ari Seff, Yinda Zhang, Shuran Song, Thomas Funkhouser, and Jianxiong Xiao. Lsun: Construction of a large-scale image dataset using deep learning with humans in the loop. *arXiv:1506.03365*, 2015. 6
- [68] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *CVPR*, 2018. 6, 13

- [69] Silvia Zuffi, Angjoo Kanazawa, Tanya Berger-Wolf, and Michael J Black. Three-d safari: Learning to estimate zebra pose, shape, and texture from images” in the wild”. In *ICCV*, 2019. [2](#), [3](#)
- [70] Silvia Zuffi, Angjoo Kanazawa, David W Jacobs, and Michael J Black. 3d menagerie: Modeling the 3d shape and pose of animals. In *CVPR*, 2017. [2](#)

A. Additional Results

Qualitative Results. In Fig. A3 we present additional qualitative results on quadruped categories from COCO [33] and for birds from CUB [57]. We provide additional results showing full 360 degree predictions for multiple different categories on project website: mehmetaygun.github.io/saor

Part Consistency. We also compared SAOR’s surface estimates with A-CSM [26] in Fig. A1. Unlike A-CSM, our method does not use any 3D parts or 3D shape priors but is still able to capture finer details like discriminating left and right legs. A-CSM groups left and right legs as a single leg while their reference 3D template has left and right legs as a separate entity. Moreover, it mixes left-right consistency if the viewpoint changes.

Without Depth. We also demonstrate examples from a variant of our model that was trained *without* using relative depth map supervision in Fig. A2. We observe that this model is still capable of estimating detailed 3D shapes with accurate viewpoints and similar textures as the full model. However, the model trained without depth maps tends to produce wider shapes compared to the full model. Quantitative results for our model without relative depth are available in Table 2 in the main paper.

Limitations. We showcase some failure cases of our method in Fig. A4. Our method fails when the animal is captured from the back, as there is insufficient data available from that angle in the training sets. Note, methods such as [60] partially address this by using alternative training data that includes image sequences from video. Furthermore, when there is also partial visibility (e.g. only the head is visible), our method produces less meaningful results as our architecture does not explicitly model occlusion.

Part Ablations. We conducted an additional ablation experiment on the number of parts used for horses. Results are provided in Table A1. Notably, the PCK scores do not significantly vary with different numbers of parts. Therefore, for all other experiments, we used 12 parts.

Number of Parts	6	12	24
PCK	34.5	34.7	34.1

Table A1: Keypoint transfer results on Pascal horses [8] where the number of parts are varied.

B. Additional Implementation Details

B.1. Data Pre-Processing

When constructing our training datasets, we run a general-purpose animal object detector [2] and eliminate objects that if any of the following criteria hold: i) the confidence of the detection is less than 0.8, ii) the minimum

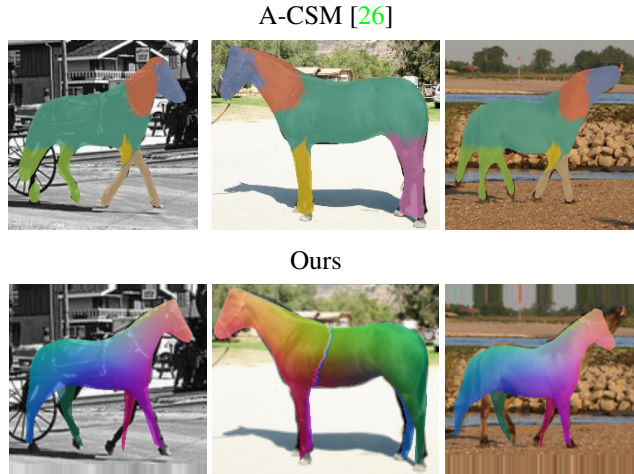


Figure A1: Comparison with A-CSM [26] on horses using example images from their paper. Even though A-CSM uses a 3D template with pre-defined fixed parts, it still maps left and right legs to the same leg in the template and the legs are not consistent across viewpoints (i.e. the part assignment is different in the top row depending on whether the horse is facing left or right). In contrast, despite not using any 3D object priors at training time, our method is much more consistent in its assignment. However, it does mistake one of the left for the the horse’s tail in the final column.

side of the bounding box is less than 32 pixels, iii) the maximum side of the bounding box is less than 128 pixels, and iv) there is no margin greater than 10 pixels on all sides of the bounding box.

We then automatically extract segmentation masks using PointRend [23] pre-trained on COCO [33]. As PointRend estimates multiple segmentation masks for each instance present in the image, we pick the segmentation mask that overlaps the most with the detected bounding box from the general-purpose detector. We automatically estimate the relative monocular depth using the transformer-based Midas [45, 46], using their Large DPT model.

To obtain cluster centers for the balanced sampling step in Section 3.3 in the main paper, we resize the estimated segmentation masks to 32×32 , and cluster the 1024-dimensional vectors into 10 clusters using a Gaussian mixture model in all of our experiments. Visualization of cluster centers of various animals can be found in Figure A5.

B.2. Architecture

We use a ResNet-50 [12] as our image encoder f_{enc} in our CUB[57] experiments and the smaller ResNet-18 in quadruped animal experiments. This is in contrast to much larger ViT-based backbones used in other work [60]. We initialize these encoders from scratch, i.e. no supervised or self-supervised pre-training is used. The architecture details

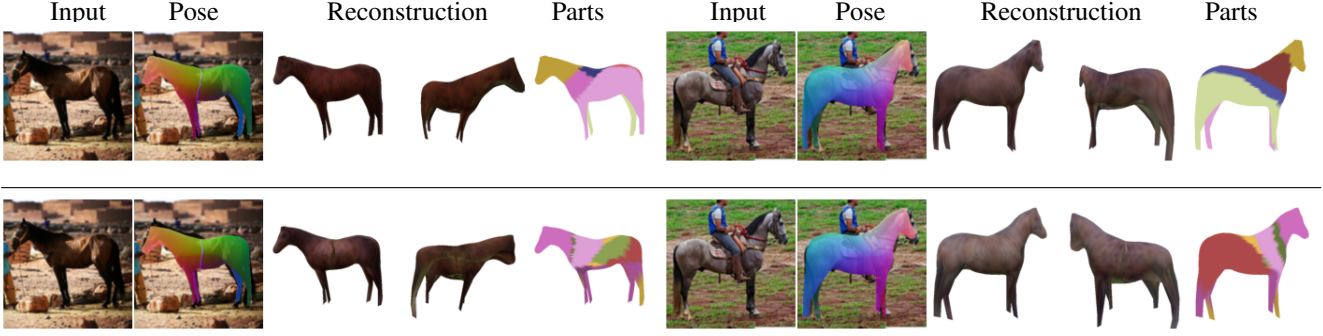


Figure A2: Comparison of models trained with relative depth supervision (top) and without (bottom). Our model trained without depth also estimates detailed 3D shapes with the correct viewpoint. However, the 3D predictions are marginally worse as it produces slightly wider 3D shapes.

Layer	Input	Output	Dim
Linear (3,512)	S°	l_x	$N \times 512$
Linear (512,512)	ϕ_{im}	l_z	512
2 x Linear (512,128)	$l_x + l_z$	L	$N \times 128$
Linear (128,3)	l	D	$N \times 3$

Table A2: Architecture details of our Deformation Net f_d .

Layer	Input	Output	Dim
Linear (3,512)	S°	l_x	$N \times 512$
Linear (512,512)	ϕ_{im}	l_z	512
Linear (512,128)	$l_x + l_z$	$N \times 128$	
Linear (128,128)	$l_x + l_z$	L	$N \times 128$
Linear (128,K)	L	W	$N \times K$
K x Linear (512, 9)	ϕ_{enc}	π	$K \times 9$

Table A3: Architecture details of our Articulation Net f_a . K is the number of parts and N is the number of vertices.

Layer	Input	Output	Dim
Linear (512,512)	ϕ_{im}	L	$512 \times 1 \times 1$
Upsample	L	L_{up}	$512 \times 4 \times 4$
Upsample + Conv2D	L_{up}		$256 \times 8 \times 8$
Upsample + Conv2D			$128 \times 16 \times 16$
Upsample + Conv2D			$64 \times 32 \times 32$
Upsample + Conv2D			$32 \times 64 \times 64$
Upsample + Conv2D			$16 \times 128 \times 128$
Conv2D		T	$3 \times 128 \times 128$

Table A4: Architecture details of our Texture Net f_t .

are presented in the following tables: deformation network f_d in Table A2, articulation network f_a in Table A3, texture network f_t in Table A4, and pose network f_p in Table A5.

Layer	Input	Output	Dim
1 x Linear (512,128)	ϕ_{im}	L	128
C x Linear (128,6)	L	r_p, t_p	128
Linear (128,C)	L	α	128

Table A5: Architecture details of our Pose Net f_p . C is the number of cameras, and α is the associated scores for each camera as in [60].

B.3. Training Losses

Here we describe the training losses from the main paper in more detail. The appearance loss is a combination of an RGB and perceptual loss [68]. $\mathcal{L}_{appr} = \lambda_{rgb}\mathcal{L}_{rgb} + \lambda_{percp}\mathcal{L}_{percp}$. These terms are defined below,

$$\mathcal{L}_{rgb} = \left\| \sum_{i,j} I_{i,j} - \hat{I}_{i,j} \right\|_2, \quad (4)$$

$$\mathcal{L}_{percp} = \left\| \phi_p(I_{i,j}) - \phi_p(\hat{I}_{i,j}) \right\|_2, \quad (5)$$

where ϕ_p is a function that extracts features from different layers of the VGG-16 [51] network.

The mask loss is calculated based on the difference between the automatically generated ground truth segmentation mask M and the estimated mask $\hat{M}$ derived from our predicted 3D shape,

$$\mathcal{L}_{mask} = \lambda_{mask} \sum_{i,j} \|M_{i,j} - \hat{M}_{i,j}\|_2. \quad (6)$$

Likewise, the depth loss is computed using the automatically generated relative depth D and the estimated depth $\hat{D}$ from the shape,

$$\mathcal{L}_{depth} = \lambda_{depth} \sum_{i,j} \|D_{i,j} - \hat{D}_{i,j}\|_2. \quad (7)$$



Figure A3: Additional qualitative results on quadrupeds and birds. Note, the part assignment displays the part with the highest probability for each vertex, but in practice the articulation for each vertex can be explained by a linear combination of multiple parts.

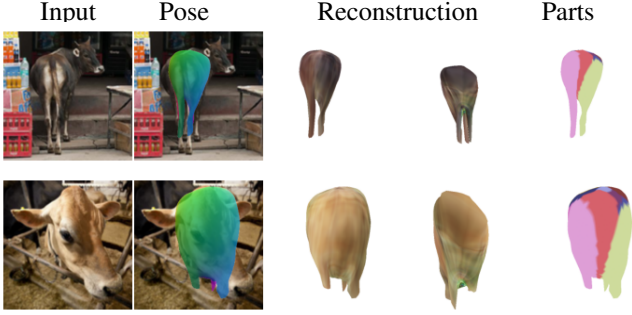


Figure A4: Failure cases on cows. When the pose is very different than the typical ones present in the training set (top) or there is too much occlusion (bottom) our method fails to produce a sensible shape estimate.

Our swap loss is a combination of the RGB and mask loss between the input image I and swapped image I^{sw} ,

$$\mathcal{L}_{swap} = \lambda_{swap} \mathcal{L}_{mask}(I, I^{sw}) + \mathcal{L}_{rgb}(I, I^{sw}). \quad (8)$$

Finally, we also employ part regularization on the part assignment matrix W to encourage equal-sized parts,

$$\mathcal{L}_{part} = \lambda_{part} \sum_k \left(\left(\sum_i W_{i,k} \right) - N/K \right) \quad (9)$$

where N is the number of vertices in the mesh and K is the number of parts. We also apply 3D regularization on the 3D shape, $\mathcal{L}_{smooth} = \lambda_{smooth} \sum LS$, where L is the laplacian of shape S and $\mathcal{L}_{normal}$ which is defined below,

$$\mathcal{L}_{normal} = \lambda_{normal} \sum_{f_i, f_j \in \Omega} 1 - \frac{\mathbf{n}_i \cdot \mathbf{n}_j}{\|\mathbf{n}_i\| \cdot \|\mathbf{n}_j\|} \quad (10)$$

here Ω is the set that includes all the pairs of faces that are neighbours, and $\mathbf{n}_i$ is the normal vector of the face f_i .

The final regularization term is defined as,

$$\mathcal{L}_{reg} = \lambda_{part} \mathcal{L}_{part} + \lambda_{smooth} \mathcal{L}_{smooth} + \lambda_{normal} \mathcal{L}_{normal}. \quad (11)$$

We present the weights used in our experiments for each loss in Table A6.

B.4. Training

We trained the Bird and Horse models from scratch. Each was trained for 500 epochs. In the first 100 epochs we only learn deformation, and then enable articulation afterward. In the case of other non-horse quadrupeds, we fine-tune models for about 200 epochs starting from the horse model. We utilize Adam [22] with a fixed learning rate for optimizing our networks. We present all the hyperparameters in Table A6.

Parameter	Value - Range
Optimization	
Optimizer	Adam
Learning Rate	1e-4
Batch Size	96
Epochs	500
Image Size	128 × 128
Mesh	
Number of Vertices	2562
Number of Faces	5120
UV Image Size	64 × 128 × 3
Number of Parts	12
Initial Position	(0,0,0)
Camera	
Translation Range	(-0.5, 0.5)
Azim Range	(-180, 180)
Elev Range	(-15, 30)
Roll Range	(-30, 30)
FOV	30
Number of Cameras	4
Loss Weights	
λ_{rgb}	1
λ_{percp}	10
λ_{mask}	1
λ_{depth}	1
λ_{swap}	1
λ_{smooth}	0.1
λ_{normal}	0.1
λ_{part}	1
λ_{pose}	0.05

Table A6: Training hyperparameters.

Our simplified swap loss leads to easy hyper-parameter selection compared to Unicorn [40]. For instance, in their swap loss term, the following parameters need to be decided: i) feature bank size, ii) minimum and maximum viewpoint difference, and iii) number of bins to divide samples in the feature bank depending on the viewpoint. Moreover, they need to do multistage training where they increase the latent dimensions for the shape and texture codes to obtain similar shapes during training. Here the number of stages and the dimension of latent codes in each stage are also hyperparameters. In our method, we eliminated all of these hyperparameters. Moreover, as we do not use all of the hypotheses cameras to estimate loss during a forward pass as in [60] and as a result of our simplified swap loss, model training is six times faster than Unicorn, as they use six cameras during training, for the same number of epochs.

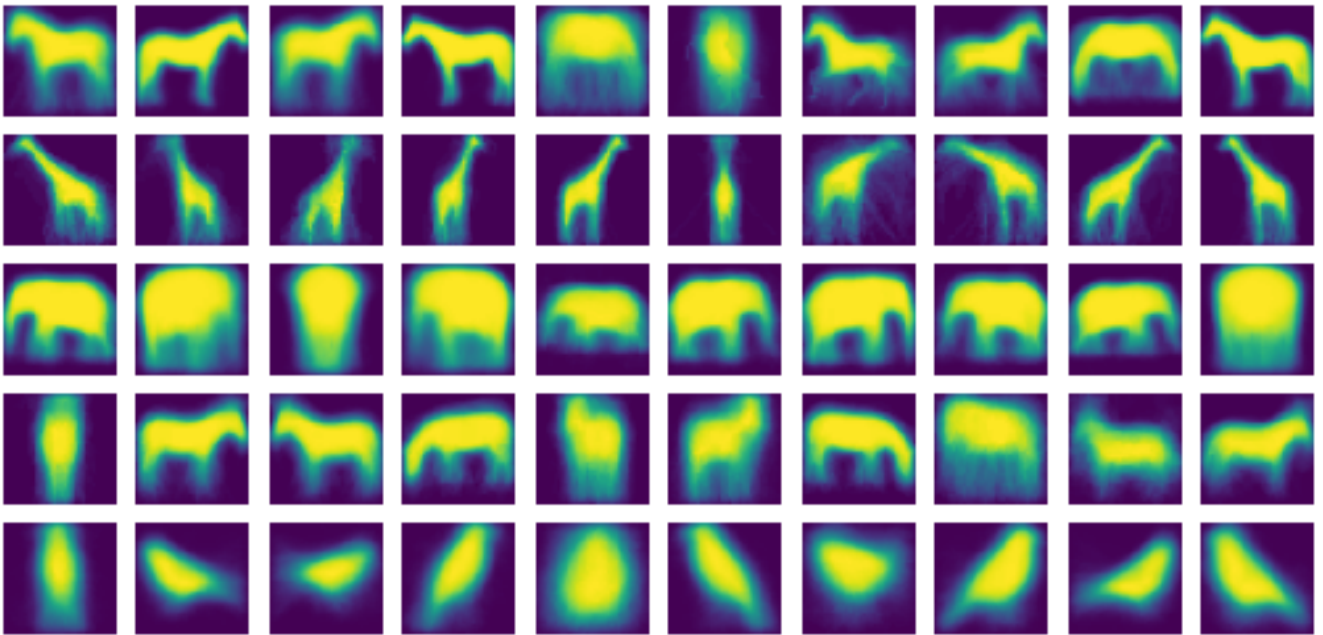


Figure A5: Visualization of the cluster centers obtained from estimated silhouettes of various animal categories used in our balanced sampling. We observe that these cluster centers broadly capture the dominant viewpoints of each object category. Top to bottom: horse, giraffe, elephant, zebra, and bird.